

Ledger

Full Build Prompt for Google Antigravity — AI-Integrated Insurance Claims Underwriting Platform

This document is a complete, self-contained project brief intended to be pasted into Antigravity (or handed to it as a reference document) to build the entire application end-to-end: frontend, backend, data model, AI decision layer, AWS integrations, and industry-grade workflow features. It is written as a single high-level prompt organized into sections so Antigravity can plan and execute the build in stages.

1. Project Overview

Build a production-oriented, full-stack Insurance Claims Underwriting Platform called “Ledger.” Claimants submit insurance claims with supporting documents through a web form. The system automatically extracts data from documents, computes a rule-based and AI-assisted risk score, generates a case summary and recommendation using an LLM, and presents everything in a unified underwriter dashboard so a decision that currently takes 30-40 days can be made in minutes with more accuracy and less manual reading of raw documents.

Primary users

- **Claimant** — submits a claim, uploads documents, tracks status.
- **Underwriter** — reviews cases in a ledger dashboard, opens a case, reads the AI summary and risk flags, and approves, rejects, or escalates.
- **Senior Underwriter / Committee** — reviews claims above a monetary threshold that require second-level approval.
- **Admin** — manages users, roles, and views aggregate analytics.

2. Tech Stack

Application layer

- Frontend: React + Tailwind CSS, fully responsive down to mobile.
- Backend: FastAPI (Python) or Node/Express — expose a clean REST API.
- Operational database: MongoDB or PostgreSQL/Aurora for live claim records, case status, and underwriter decisions. This is the transactional store the dashboard reads and writes against.
- Auth: AWS Cognito for claimant and underwriter login, mapped to IAM roles and app-level role-based access control (claimant / underwriter / senior underwriter / admin).

AWS data and storage layer (current stack, keep and build around it)

- **Amazon S3** — raw document storage (claim forms, bills, medical reports, photos, FIR copies).
- **AWS KMS** — server-side encryption (SSE-KMS) on all S3 objects and sensitive database fields.
- **AWS Glue** — ETL jobs that transform processed claim data from the operational database into an analytics-ready format in S3 / the Glue Data Catalog.
- **Amazon Athena** — SQL query layer over the Glue Data Catalog for ad-hoc and scheduled analytics queries.
- **Amazon QuickSight** — BI dashboard fed by Athena, embedded into the app's Analytics view for management-level reporting.
- **AWS IAM** — least-privilege roles scoping access to S3, Glue, Athena, QuickSight, and every other AWS service in this stack; separate roles for claimant, underwriter, admin, and CI/CD.

AWS intelligence and workflow layer (new additions to include)

- **AWS Textract** — OCR and structured field extraction from uploaded claim documents (bill amounts, dates, names, diagnosis codes) so underwriters don't read raw PDFs.
- **AWS Comprehend** (or **Comprehend Medical** for health claims) — entity extraction and NLP on claim descriptions and extracted document text.
- **AWS Bedrock** (Claude) — the AI decision layer: generates the case summary and Approve / Investigate Further / Reject recommendation with reasoning. Called server-side only; never expose model credentials to the frontend.
- **AWS Fraud Detector** — trained fraud risk model, blended with the rule-based risk score described in Section 5.
- **AWS Step Functions** — orchestrates the claim-processing pipeline (upload > Textract extraction > fraud/risk scoring > AI summary > status update) as an auditable state machine.
- **Amazon SQS** — decouples claim submission from downstream processing so the claimant-facing form responds instantly instead of waiting on extraction and AI calls.
- **AWS CloudTrail** — audit log of every underwriter decision and sensitive data access, required for regulatory review.
- **Amazon Macie** — scans S3 documents for sensitive personal/medical data (PII) automatically.
- **Amazon SES / SNS** — email/SMS notifications to claimants on status changes.

Note for Antigravity: structure all AWS integrations behind clean service interfaces so they can be mocked or stubbed for local development, and only wired to real AWS resources when credentials are provided via environment variables. Never hardcode any AWS key, secret, or Bedrock model ID.

3. Design Direction

Palette

Ink (primary / headers)	#14213D
Paper (background)	#F7F6F1
Steel (secondary text)	#5C6B73
Amber (pending / flags)	#C8862A
Sage (approved / positive)	#3E6E5B
Rust (rejected / high risk)	#A6394A

Typography and feel

- Headings: Fraunces (serif). Body: Inter. Case IDs, amounts, and data values: IBM Plex Mono.
- Overall feel: a professional ledger / insurance-office aesthetic — trustworthy and document-like, not a generic SaaS dashboard template.
- Status shown as rotated rubber-stamp style badges: SUBMITTED, UNDER REVIEW, APPROVED, REJECTED, ESCALATED.
- Sidebar navigation: “File a Claim,” “Underwriter Ledger,” “Analytics,” “Admin” (role-gated).

4. Core Data Model

Claim

```
id, claimantName, policyNumber, policyType (Health / Motor / Life / Travel / Property),  
sumInsured, policyStartDate, incidentDate, claimAmount, contactNumber, description,  
documents (array of {name, type, s3Key, extractedFields}), status (submitted / review /  
approved / rejected / escalated), riskScore, riskFlags, fraudDetectorScore, aiSummary,  
aiRecommendation, assignedUnderwriterId, submittedAt, decidedAt, decidedBy, auditTrail  
(array of {action, actor, timestamp}).
```

Supporting entities

- **PolicyholderHistory** — derived view: all claims sharing a policy number.
- **User** — id, name, role (claimant / underwriter / senior_underwriter / admin), email, Cognito identity.
- **Notification** — id, claimId, recipient, channel (email/sms), message, sentAt.
- **Disbursement** — claimId, approvedAmount, payoutMethod, bankDetailsRef, status, disbursedAt.

5. Risk Scoring Rules

Compute a rule-based risk score (0-100) server-side, recomputed whenever a claim is fetched. Blend it with the AWS Fraud Detector score as a weighted average when available.

- Incident within 30 days of policy start → +40 risk, flag “possible waiting-period violation.”
- Incident within 90 days of policy start → +15 risk, flag “within early-claim window.”
- Claim amount > 90% of sum insured → +30 risk, flag “unusually high vs sum insured.”
- Claim amount > 60% of sum insured → +10 risk.
- Fewer than 2 supporting documents attached → +15 risk, flag “may be insufficient for verification.”
- Description under 12 words → +10 risk, flag “very brief, may need follow-up.”
- Cap total at 100. Risk level bands: 0-19 low (sage/green), 20-49 medium (amber), 50+ high (rust/red).

Also evaluate the claim against the policy's own terms and conditions where structured policy rules exist (coverage inclusions/exclusions, sub-limits per claim type, waiting periods per rider, co-pay clauses). Where a full policy-rules engine isn't yet built, use AWS Bedrock with retrieval-augmented generation over the policy document text so the AI recommendation cites the specific clause it relied on.

6. API Endpoints

- **POST /api/claims** — create claim, upload documents to S3 (KMS-encrypted), enqueue processing via SQS.
- **GET /api/claims** — list claims, filterable by status, policyType, assigned underwriter, and free-text search.
- **GET /api/claims/{id}** — full claim detail + policyholder history + risk assessment.
- **PATCH /api/claims/{id}** — update status/decision; writes a CloudTrail-logged audit entry.
- **POST /api/claims/{id}/process** — trigger Textract extraction, risk scoring, and AI summary generation (or run automatically via Step Functions on submission).
- **POST /api/claims/{id}/escalate** — send a claim to senior-underwriter review when it exceeds the auto-approval threshold.
- **POST /api/claims/{id}/disburse** — record payout details once a claim is approved.
- **GET /api/analytics/embed-url** — generate a QuickSight embedded dashboard URL scoped to the requesting user's IAM permissions.

7. Pages / Views

7.1 File a Claim (claimant form)

- Fields: Claimant Name, Policy Number, Policy Type (dropdown), Sum Insured, Policy Start Date, Incident Date, Claim Amount, Contact Number, Description (textarea).
- Repeatable document upload (drag-and-drop to S3), each tagged with a type: Bill/Invoice, ID Proof, Medical Report, Police Report (FIR), Photos, Other.
- Explicit consent/privacy notice checkbox before submission (required for legal validity).
- On submit: files uploaded to S3 (KMS-encrypted), claim record created with status “submitted,” processing pipeline triggered, claimant redirected to a status-tracking page.

7.2 Underwriter Ledger (dashboard)

- Table columns: Case ID, Claimant + Policy Number, Policy Type, Claim Amount, Status (stamp badge), Risk Score (colored dot + number out of 100), assigned underwriter.
- Filters: status, policy type, assigned underwriter, free-text search by name/policy number.
- Auto-assignment: new claims distributed across underwriters based on current workload and, where tagged, policy-type specialty.
- Role-gated: visible only to underwriter, senior_underwriter, and admin roles.

7.3 Case Detail

- Full claim data: amount, sum insured, dates, contact, description.

- Extracted document fields (from Textract) alongside the original document chips with S3-backed previews.
- Risk Assessment panel: score, each flag with an icon (check / warning / alert) and explanation text, plus the Fraud Detector score if available.
- Policyholder History panel: other claims on the same policy number, with their status and outcome.
- AI Case Summary panel: Bedrock-generated 2-3 sentence summary and a clear recommendation (Approve / Investigate Further / Reject) with reasoning tied to specific flags and policy clauses.
- Decision panel: status dropdown (Submitted / Under Review / Approved / Rejected / Escalated) and Save, which writes an audit entry. Claims above the configured monetary threshold require escalation to a senior underwriter instead of a direct decision.
- Investigator assignment: when status is set to "Investigate Further," allow assigning a surveyor/investigator and attaching their findings once returned.

7.4 Analytics (embedded QuickSight)

- Embedded QuickSight dashboard: claims by status over time, average decision turnaround (the core metric proving the 30-40 day improvement), risk score distribution, claims by policy type/region, approval vs rejection rate, underwriter workload and performance.

7.5 Admin

- User and role management (claimant / underwriter / senior_underwriter / admin).
- View CloudTrail-backed audit log of all decisions and data access.
- Configure the auto-approval / escalation monetary threshold per policy type.

8. Industry-Grade Features Beyond the Core MVP

Include these so the platform reflects how a real insurer's claims workflow actually operates, not just a demo of the AI decision loop:

- **Multi-level approval** — claims above a configurable amount threshold require senior-underwriter or committee sign-off, not a single reviewer's decision.
- **SLA / turnaround tracking** — timestamp every stage of a claim's lifecycle and surface average turnaround time on the Analytics dashboard, since reducing the 30-40 day cycle is the whole premise of the project.
- **Appeals / re-submission flow** — a claimant can respond to a rejection or an “information requested” status with additional documents, re-opening the case rather than starting over.
- **Claimant notifications** — email/SMS via SES/SNS on every status change.
- **External verification hooks** — pluggable integrations for hospital-network APIs (health), surveyor/garage reports (motor), and a cross-insurer claims history check for fraud detection (a claimant's fraud pattern is often invisible from one insurer's data alone).
- **E-signature / consent capture** — legally binding acknowledgement at both claim submission and final decision communication.
- **Investigator assignment workflow** — a real next step for claims marked “Investigate Further,” not a dead-end status.
- **Payout / disbursement layer** — bank details capture, payment gateway or disbursement ledger integration once a claim is approved.
- **Reserve / liability tracking** — insurers track money reserved against each open claim, not only the final payout amount.
- **Data retention policy** — automated retention and deletion rules reflecting typical insurance regulatory requirements (often 7+ years).
- **Document versioning** — if a claimant re-uploads a corrected document, keep history rather than overwriting the original.
- **Underwriter workload balancing** — auto-assign new claims based on current caseload and, where tagged, specialty.

9. Seed Data

Seed the database with these six sample claims on first run:

1. **Ramesh Kumar** — POL-88213, Health, sum insured ■500,000, policy start 620 days ago, incident 9 days ago, claim amount ■145,000. “Hospitalized for 4 days for acute appendicitis, underwent laparoscopic surgery at Apollo Hospital.” Documents: Hospital discharge summary, Final bill, Aadhaar card. Status: Submitted.
2. **Priya Nair** — POL-91027, Motor, sum insured ■800,000, policy start 14 days ago, incident 2 days ago, claim amount ■610,000. “Car collided with a divider on the highway, front end damaged.” Documents: Photos of damage. Status: Under review.

3. Arjun Reddy — POL-77410, Health, sum insured ■300,000, policy start 900 days ago, incident 30 days ago, claim amount ■42,000. "Outpatient treatment for dengue fever, blood tests and medication over one week, fully recovered." Documents: Lab reports, Pharmacy bills, Doctor prescription. Status: Approved.

4. Sunita Verma — POL-60312, Travel, sum insured ■150,000, policy start 45 days ago, incident 40 days ago, claim amount ■138,000. "Trip cancelled last minute." Documents: None. Status: Rejected.

5. Mohammed Iqbal — POL-53390, Property, sum insured ■2,000,000, policy start 1200 days ago, incident 5 days ago, claim amount ■380,000. "Kitchen fire caused by electrical short circuit damaged cabinetry, appliances, and part of the ceiling. Fire department report filed." Documents: Fire department report, Repair estimate, Damage photos, Property ownership proof. Status: Submitted.

6. Ramesh Kumar — POL-88213, Health, sum insured ■500,000, policy start 620 days ago, incident 200 days ago, claim amount ■60,000. "Routine cataract surgery, single day procedure, no complications." Documents: Hospital bill, Discharge summary. Status: Approved.

Claim 6 is a second claim on the same policy as Claim 1 (same claimant, POL-88213) so the Policyholder History panel has real data to demonstrate.

10. Non-Functional Requirements

- All AWS credentials and secrets in environment variables; never hardcoded or exposed client-side.
- IAM roles follow least privilege: claimant can only write their own claim and read their own status; underwriter can read/write assigned claims; senior_underwriter can approve escalated claims; admin has Glue/Athena/QuickSight and user-management access.
- All S3 objects encrypted at rest via KMS; sensitive database fields encrypted as well.
- Every underwriter decision and every access to a claim's sensitive data is written to the CloudTrail-backed audit log.
- Fully responsive down to mobile.
- Structure AWS integrations (Textract, Bedrock, Fraud Detector, Step Functions, SQS, SES/SNS, Macie) behind swappable service interfaces so the app can run against mocks locally and against real AWS resources in a deployed environment.
- This is a strong prototype / portfolio-grade build, not a certified production insurance system — prioritize clarity, demoability, and clean architecture over exhaustive edge-case handling.

11. Build Order (suggested phases)

- **Phase 1** — Core app: claim submission form, underwriter ledger, case detail view, MongoDB/Postgres storage, seed data, rule-based risk scoring.
- **Phase 2** — AWS storage integration: S3 + KMS for documents, IAM roles, Cognito auth.
- **Phase 3** — AI layer: Bedrock case summary/recommendation, Textract document extraction, Fraud Detector risk blending.
- **Phase 4** — Orchestration and reliability: Step Functions pipeline, SQS decoupling, SES/SNS notifications, CloudTrail audit logging, Macie PII scanning.
- **Phase 5** — Analytics: Glue ETL jobs, Athena queries, embedded QuickSight dashboard.
- **Phase 6** — Industry-grade workflow: multi-level approval, escalation, appeals, investigator assignment, disbursement, workload balancing, admin panel.